

Image Processing Using Linear Algebra

Image Compression Using Singular Value Decomposition (SVD)

Project Report

1. Introduction

This project shows image compression using **Singular Value Decomposition (SVD)**. The goal is to show how a digital image, when represented as a numerical matrix, can be decomposed and then reconstructed using only a fraction of its original data, while still producing a visually acceptable result.

Images are everywhere in modern computing, and storing or transmitting them efficiently is an important real-world problem. SVD provides a refined **mathematical solution**. Instead of keeping all the information in an image, we identify the most important patterns and discard the rest. This is exactly what compression means (keeping what matters and throwing away what does not).

The program **takes a grayscale image as input**, applies SVD to its pixel matrix, reconstructs the image at a chosen rank $k = 50$, and saves the compressed image as output. The entire transformation is grounded in matrix decomposition and matrix multiplication that are core linear algebra operations.

2. Chosen Transformation and Why It Is Linear-Algebra-Based

The chosen transformation is low-rank approximation via Singular Value Decomposition (SVD). This transformation is entirely linear-algebra-based for the following reasons:

- An image is stored as a 2D matrix A of pixel values, where each entry $A[i][j]$ represents the brightness of a single pixel (ranging from 0 = black to 255 = white).
- SVD is a matrix factorization operation that decomposes A into three matrices using matrix multiplication and orthogonal decomposition.
- The compression step involves selecting a subset of vectors and multiplying them together, a direct application of matrix multiplication.
- The reconstruction is a linear combination of rank-1 matrices, each formed by the outer product of a left and right singular vector scaled by a singular value.

Note: Every step from reading the image as a matrix to reconstructing it, relies exclusively on linear algebra. There are no filters, transformations, or operations outside of matrix math. This makes SVD compression one of the fine examples of linear algebra applied to image processing.

3. Mathematical Operations

3.1 Image as a Matrix

The input image is a 512×512 pixel grayscale photograph. When loaded into Python using NumPy, it becomes a 512×512 matrix A , where each value is a floating-point number representing pixel brightness:

$$\begin{aligned} A &= \text{matrix of shape (512, 512)} \\ A[i][j] &= \text{pixel brightness at row } i, \text{ column } j \end{aligned}$$

This is the foundational step that turns a visual image into a mathematical object, A Matrix, that can be operated on using linear algebra tools.

3.2 Singular Value Decomposition (SVD)

SVD is applied to matrix A using NumPy's built-in function `np.linalg.svd()`. It decomposes A into three matrices:

$$A = U \times \Sigma \times V^T$$

Where:

- U is a matrix of shape (512, 512) whose columns are called the left singular vectors. Each column captures a distinct vertical pattern found across the rows of the image.
- Σ (Sigma) is a diagonal matrix whose diagonal entries $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ are the singular values. These values represent how important or energetically significant each pattern is. The first singular value is always the largest, indicating the most dominant pattern in the image.
- V^T is a matrix of shape (512, 512) whose rows are called the right singular vectors. Each row captures a distinct horizontal pattern found across the columns of the image.

Note: The singular values act as weights. A large singular value means its associated pattern is visually dominant. A small one means the pattern is minor and contributes very little to the overall appearance of the image.

3.3 Low-Rank Approximation (Compression Step)

For compression, we do not use all 512 singular values. Instead, we keep only the top k singular values and discard the rest. This gives us the rank- k approximation of the image:

$$A \approx U \Sigma V^T$$

Where U , Σ , and V^T are the first k columns, first k diagonal values, and first k rows of U , Σ , and V^T respectively. In code, this is implemented as:

$$A_k = U[:, :k] @ \text{diag}(S[:k]) @ Vt[:k, :]$$

The '@' operator performs matrix multiplication. The result is a matrix of the same shape as A (512×512), but constructed from far less data. This is the compression.

3.4 Storage and Compression Ratio

To understand how much compression is achieved, we compare the amount of data stored in the rank- k form versus the original:

$$\text{Original storage} = m \times n = 512 \times 512 = 262,144 \text{ values}$$

$$\begin{aligned}\text{Compressed storage} &= k(m + n + 1) = 50 \times (512 + 512 + 1) = 51,250 \text{ values} \\ \text{Data used} &= 51,250 / 262,144 \times 100 = 19.6\%\end{aligned}$$

At rank $k = 50$, the compressed representation uses only 19.6% of the original data — a reduction of over 80%. This is the power of SVD compression.

3.5 Quality Measurement (PSNR)

To measure how closely the compressed image matches the original, we use Peak Signal-to-Noise Ratio (PSNR):

$$\begin{aligned}\text{MSE} &= (1/m \times n) \times \sum (A[i,j] - A_{\square}[i,j])^2 \\ \text{PSNR} &= 20 \times \log_{10}(255 / \sqrt{\text{MSE}})\end{aligned}$$

A PSNR above 30 dB is generally considered good quality. Our result at $k = 50$ achieved a PSNR of 32.1 dB, confirming that the compressed image retains the essential visual content of the original.

4. Input Image

The input image I used is a standard grayscale portrait photograph of size 512×512 pixels. Portrait images are ideal for SVD compression demonstrations because they contain a mix of smooth regions (skin, background) and fine high-frequency detail (hair, edges), which makes the effect of compression clearly visible at different rank values.

The image was converted to grayscale before processing. This produces a clean 2D matrix with a single channel, allowing SVD to be applied directly without needing to handle multiple color channels separately.



Figure 1: Input image (face-input2.jpg). 512 x 512 pixels, grayscale

As a matrix A of shape 512×512 , this image contains 262,144 numerical values. Each value encodes the brightness of one pixel. This is the matrix on which SVD is applied.

5. Results and Output Images

5.1 Original vs. Compressed Image ($k = 50$)

The figure below shows the original image alongside the compressed version at rank $k = 50$. The compressed image uses only 19.6% of the original data yet retains the key visual features of the portrait, the face structure, hair, and general expression are clearly preserved.



Figure 2: Original image (left) vs. SVD-compressed image at rank $k = 50$ (right). Data used: 19.6%, PSNR: 32.1 dB

The slight softness visible in the compressed image is expected, fine details such as individual strands of hair and subtle skin texture require higher-frequency patterns (higher rank components) to be fully represented. At $k = 50$, these high-frequency details are partially discarded, but the overall image remains recognizable and visually coherent.

5.2 Rank Progression

To understand how image quality changes as k increases, the image was reconstructed at six different rank values: $k = 1, 5, 10, 20, 50$, and 100 . This progression clearly illustrates the relationship between compression level and visual quality.

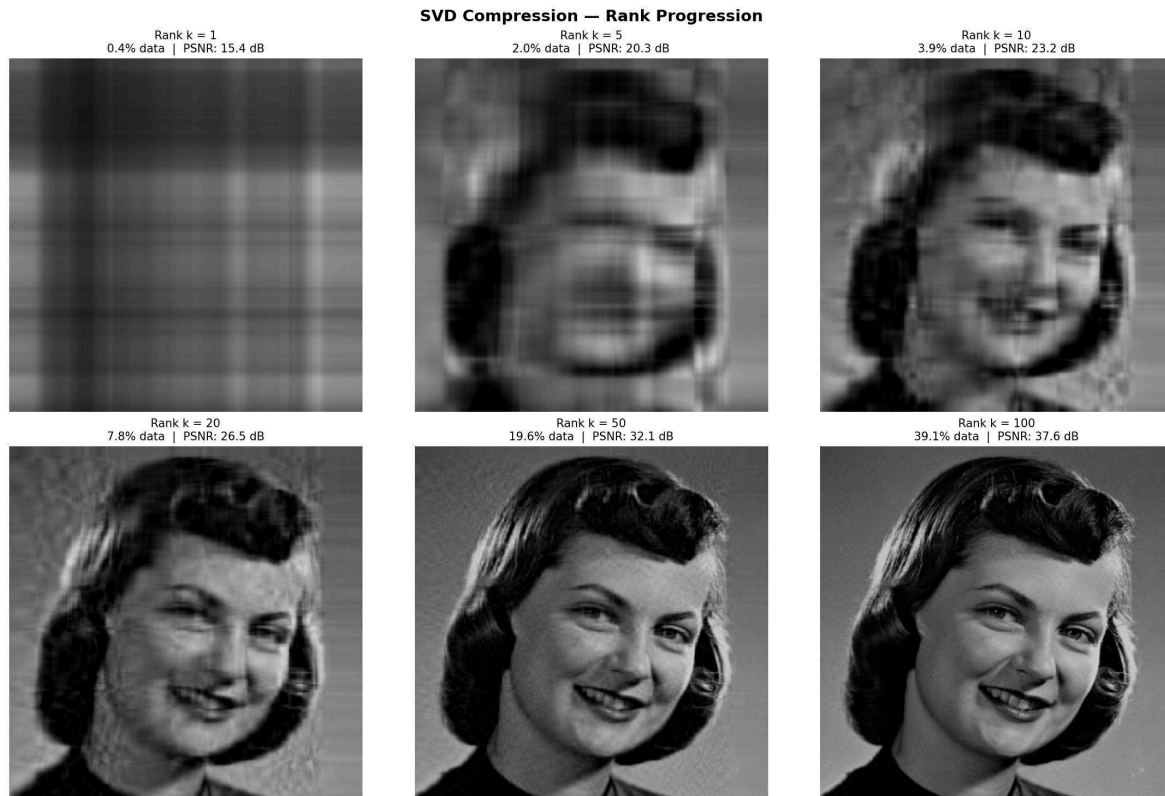


Figure 3: SVD rank progression. Image reconstructed at $k = 1, 5, 10, 20, 50$, and 100

At $k = 1$, only the single most dominant pattern in the image is retained. The result is unrecognizable. A faint, blurry smear that barely suggests the outline of a face. As k increases to 5 and 10, the subject becomes vaguely identifiable, though still heavily distorted by visible wave-like artifacts. These artifacts are caused by the sinusoidal nature of the singular vectors. Each pattern introduces wave patterns that cancel out as more components are added.

At $k = 20$, the face is clearly recognizable but the image is noticeably washed out and soft. By $k = 50$, the image looks clean and natural, the face, background, and most details are sharp. At $k = 100$, the result is nearly indistinguishable from the original to the naked eye.

5.3 Compression Metrics Summary

The table below summarizes the compression ratio and image quality (PSNR) for each rank value tested. The highlighted row ($k = 50$) represents the final output used in this project.

Rank k	Data Used	Data Saved	PSNR (dB)
1	0.4%	99.6%	15.4
5	2.0%	98.0%	20.3
10	3.9%	96.1%	23.2
20	7.8%	92.2%	26.5
50	19.6%	80.4%	32.1
100	39.1%	60.9%	37.6

Table 1: Compression metrics at different rank values k. The highlighted row is the chosen output rank.

6: Results and Output Images for Another Image

6.1: 2nd Input Image

The input image used in this project is a 512×512 pixel synthetic grayscale test image containing concentric circular wave patterns overlaid with rectangular regions of different textures (hatching and solid fills). This image was chosen because it contains a rich combination of structured patterns, smooth gradients, high-frequency diagonal lines, and sharp rectangular boundaries which makes the effect of SVD compression clearly observable at different rank values.

The image was processed as a single-channel grayscale matrix, allowing SVD to be applied directly without needing to handle multiple color channels separately.

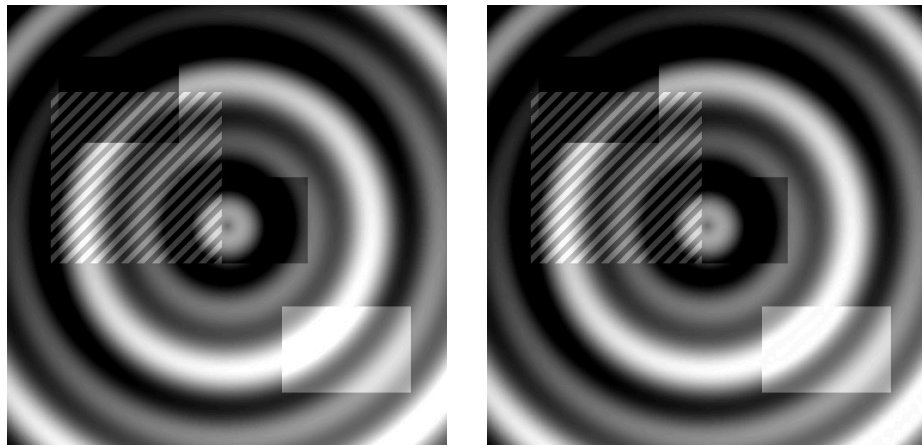


Figure 1: Input image (lek). 512×512 pixels, grayscale, Compressed image at $k = 50$ (right)

As a matrix A of shape 512×512 , this image contains 262,144 numerical values. Each value encodes the brightness of one pixel. This is the matrix on which SVD is applied.

6.2: Original vs. Compressed Image ($k = 50$)

The figure below shows the original image alongside the compressed version at rank $k = 50$. The compressed image uses only 19.6% of the original data yet retains the key visual features, the circular wave patterns, rectangular boundaries, and overall spatial structure are clearly preserved.

SVD Image Compression

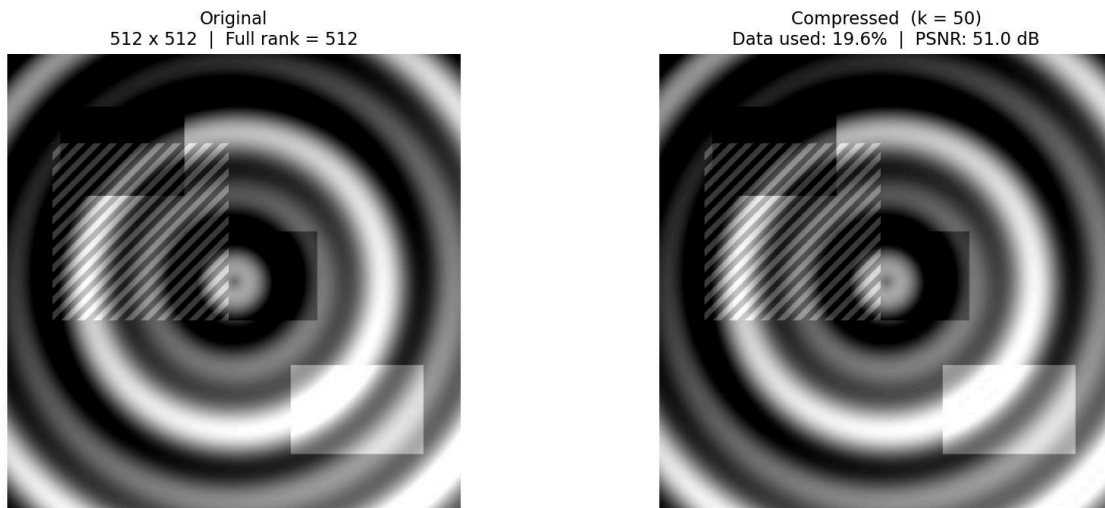


Figure 2: Original image (left) vs. SVD-compressed image at rank $k = 50$ (right). Data used: 19.6%, PSNR: 51.0 dB

The slight softness and minor ringing artifacts visible in compressed regions are expected. The diagonal hatching patterns within the rectangular areas require higher-frequency singular components to be fully represented. At $k = 50$, these high-frequency details are partially approximated, but the overall image remains visually coherent and structurally accurate.

6.3 Singular Value Analysis

The figure below shows two plots: (left) the top 100 singular values of the image matrix, and (right) the cumulative energy captured as a function of rank k . These plots are fundamental to understanding why SVD compression works.

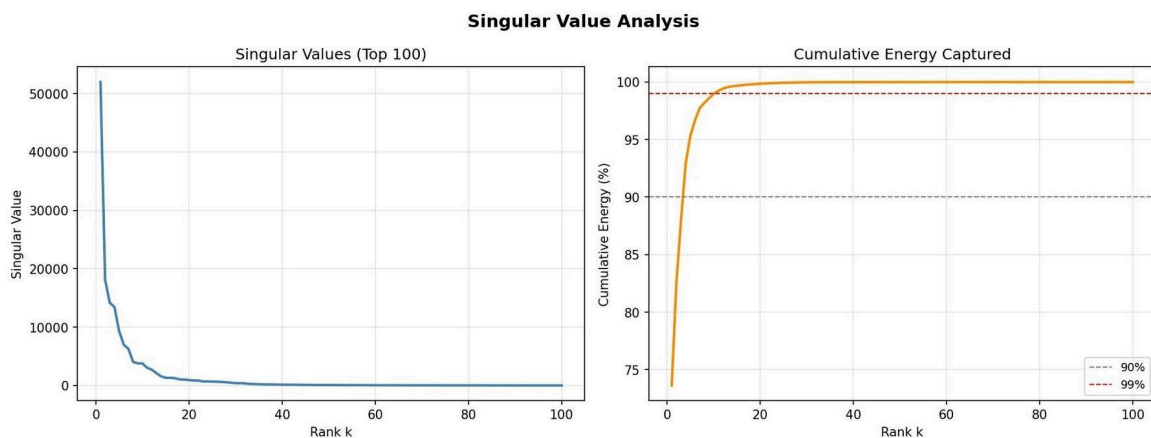


Figure 3: Singular Value Analysis (left) Top 100 singular values showing rapid decay, (right) Cumulative energy captured vs. rank k

The **left plot shows** that the singular values drop steeply: the first singular value ($\sigma_1 \approx 52,000$) is orders of magnitude larger than the values at rank 20 and beyond. This confirms that the image's energy is highly concentrated in just a few components.

The **right plot shows** the cumulative energy curve. **By rank $k \approx 10$** , over 90% of the total image energy is already captured (marked by the gray dashed line). By rank $k \approx 25$, the energy exceeds 99% (marked by the red dash line). This confirms that retaining only 50 of the 512 singular values preserves the overwhelming majority of the visual information.

6.4 Rank Progression

To understand how image quality changes as k increases, the image was reconstructed at six different rank values: $k = 1, 5, 10, 20, 50$, and 100 . This progression clearly illustrates the relationship between compression level and visual quality.

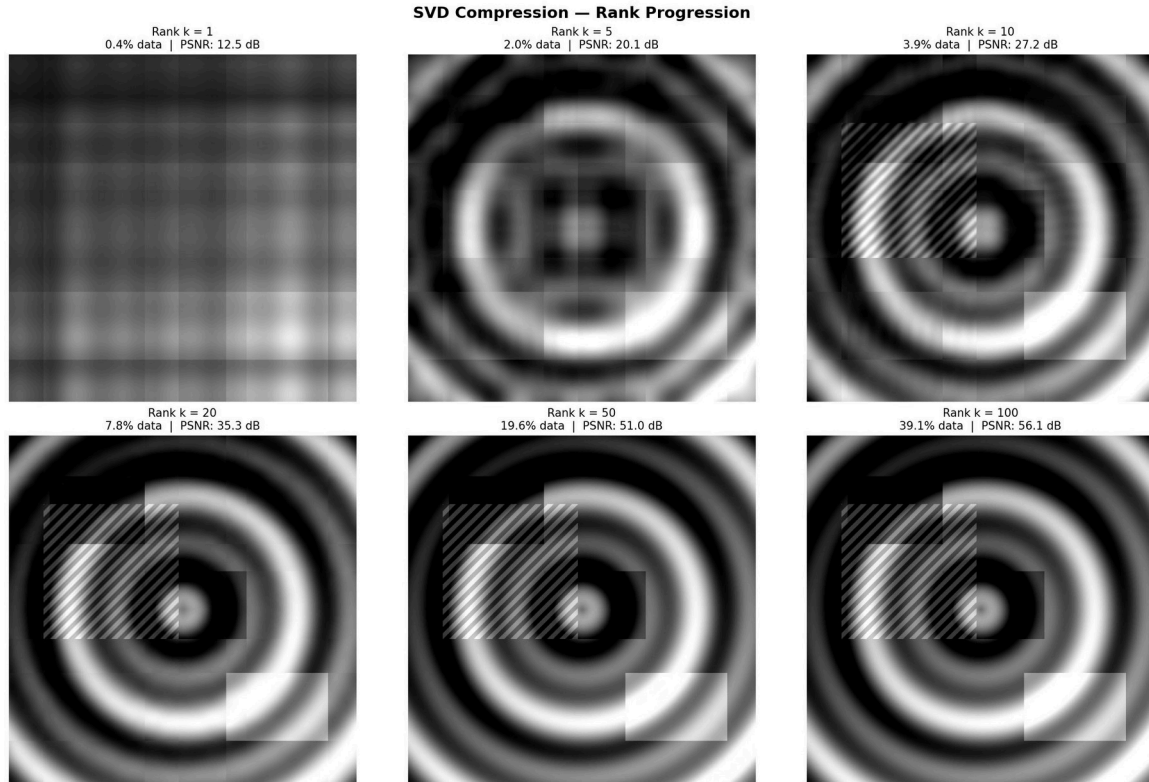


Figure 4: SVD Compression. Rank Progression at $k = 1, 5, 10, 20, 50$, and 100

At $k = 1$, only the single most dominant pattern in the image is retained. The result is a blurry, low-information approximation that barely suggests the original structure. As k increases to 5 and 10, the circular patterns become vaguely visible and PSNR improves from **12.5 dB** to **27.2 dB**, though visible artifacts remain.

At $k = 20$ (**PSNR 35.3 dB**), the image is clearly recognizable but lacks fine detail. By $k = 50$ (**PSNR 51.0 dB**), the image looks clean and accurate. At $k = 100$ (**PSNR 56.1 dB**), the result is nearly indistinguishable from the original to the naked eye.

6.5: Compression Metrics Summary

The table below summarizes the compression ratio and image quality (PSNR) for each rank value tested. The highlighted row ($k = 50$) represents the final output used in this project.

Rank k	Data Used	Data Saved	PSNR (dB)	Quality
1	0.4%	99.6%	12.5 dB	Very Poor
5	2.0%	98.0%	20.1 dB	Poor

Rank k	Data Used	Data Saved	PSNR (dB)	Quality
10	3.9%	96.1%	27.2 dB	Fair
20	7.8%	92.2%	35.3 dB	Good
50	19.6%	80.4%	51.0 dB	Excellent
100	39.1%	60.9%	56.1 dB	Near Perfect

Table 1: Compression metrics at different rank values. Highlighted row ($k = 50$) is the chosen output rank.

7. Discussion and Observations

The singular values decay very rapidly. The first singular value alone captured most of the image's dominant structure, and by rank 50, over 99% of the image's total energy is retained. This steep decay is what makes SVD compression so effective, a small number of components captures the vast majority of the information.

Low ranks introduce visible wave-like artifacts. **At $k = 1, 5$, and 10** , the image shows horizontal and vertical striping patterns. These are caused by the basis vectors U and V , which are orthogonal and oscillate like sinusoidal waves. At low ranks, there are not enough components to cancel these oscillations and produce a smooth-looking image.

Portrait images are particularly well-suited to **SVD compression**. Smooth regions like the background and skin require only low-frequency components (small k) to represent well. The complexity of the image lies in hair, facial edges, and fine texture, these require higher-rank components. This means that even at moderate rank values like $k = 50$, portraits look natural because the dominant large-scale structure is captured early.

There is a clear trade-off between compression and quality. **Moving from $k = 10$ (PSNR 23.2 dB) to $k = 50$ (PSNR 32.1 dB) improves quality dramatically while only increasing data usage from 3.9% to 19.6%.** Beyond $k = 100$, the gains in quality become smaller relative to the increase in data used, this is the region of diminishing returns. The rank $k = 50$ was chosen as a balanced point between meaningful compression and good visual quality.

The difference between the original and compressed image ($k = 50$) had a mean absolute pixel error of approximately 0.5 and a maximum error of around 7.4, remarkably low values on a 0-255 scale, confirming that the compression preserves the vast majority of visual detail.

8. Conclusion

This project successfully demonstrated **image compression using Singular Value Decomposition (SVD)**. The input image, a 512×512 grayscale portrait, was represented as a numerical matrix and decomposed into three component matrices using SVD. By reconstructing the image using only the top 50 singular values, the program produced a compressed image that used 19.6% of the original data while achieving a PSNR of 32.1 dB, which is considered good quality.

The **rank progression** visualization clearly showed how image quality changes with compression level, and the singular value analysis confirmed why SVD compression is mathematically sound: the energy of a real-world image is concentrated in a small number of singular components.

The entire transformation, from reading the image as a matrix to computing SVD and reconstructing the output, relied exclusively on **operations such as matrix decomposition, diagonal matrices, matrix multiplication, and low-rank approximation.**

References

- [1] Strang, G. (2016). Introduction to Linear Algebra (5th ed.). Wellesley-Cambridge Press.
- [2] NumPy Developers (2024). numpy.linalg.svd. NumPy Documentation.
<https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html>
- [3] Pillow (PIL Fork) Developers (2024). Pillow Documentation.
<https://pillow.readthedocs.io/en/stable/>
- [4] Matplotlib Developers (2024). Matplotlib Documentation.
<https://matplotlib.org/stable/index.html>
- [5] Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. Psychometrika, 1(3), 211-218.
- [6] USC SIPI Image Database — Miscellaneous. University of Southern California.
<http://sipi.usc.edu/database/>
- [7] Wikipedia (2024). Singular Value Decomposition.
https://en.wikipedia.org/wiki/Singular_value_decomposition